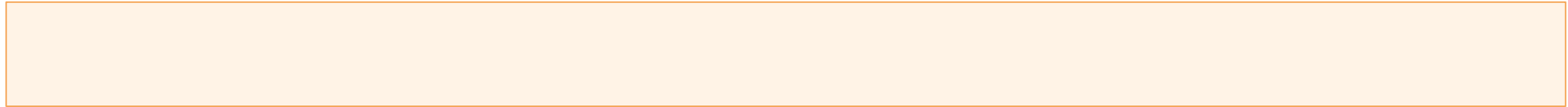


# Parsing in Compiler Design

100-slide detailed UG CSE deck • Concepts inspired by Aho, Sethi & Ullman / Aho & Ullman compiler texts



# How to use this deck

## PART 1 • Parsing overview

- Use it after lexical analysis: students should already know tokens and regular expressions.
- Each section moves from intuition → formal definitions → algorithm → trace example.
- Automata-heavy slides are included for LR parsing because LR states are automata states.
- For teaching: spend more time on FIRST/FOLLOW, LL(1) table construction and LR shift-reduce tracing.

### Teaching path

- 1 Token stream to grammar
- 2 Parse tree and derivations
- 3 LL predictive table
- 4 LR item automata
- 5 Yacc/Bison workflow

# Learning outcomes

## PART 1 • Parsing overview

- Explain the role of syntax analysis in a compiler pipeline.
- Design context-free grammars for expressions, statements and blocks.
- Compute FIRST and FOLLOW sets for LL(1) parsing.
- Construct predictive parsing tables and trace stack behavior.
- Build LR(0), SLR, LR(1) and LALR intuition using item automata.
- Write small Yacc/Bison grammars integrated with Lex/Flex scanners.

### Parser in the compiler pipeline



Parser accepts only token streams that can be generated by the programming-language grammar.

```
if (x < 10) y = y + 1;  
ID ( ID < NUM ) ID = ID + NUM ;
```

# Where parsing fits

## PART 1 • Parsing overview

- The scanner groups characters into tokens; the parser validates token sequences.
- The parser enforces the context-free structure of the programming language.
- On success, it usually builds a parse tree or abstract syntax tree.
- On failure, it should report meaningful syntax errors and recover enough to continue.

### From tokens to structure



Parser accepts only token streams that can be generated by the programming-language grammar.

```
if (x < 10) y = y + 1;
```

```
ID ( ID < NUM ) ID = ID + NUM ;
```

# Input and output of a parser

## PART 1 • Parsing overview

- Input: token stream from lexical analysis, e.g., ID, NUM, IF, WHILE, PLUS.
- Output: parse tree, AST, or calls to semantic actions.
- Side effects: syntax error messages, synchronization, recovery.
- Parser never checks all meaning; type checking and declarations are semantic-analysis tasks.

### Parser I/O

Use the example on the left to trace parser behavior.

Input

Stack

Action

# Syntax analysis: central idea

## PART 1 • Parsing overview

- Given a grammar  $G$  and input token string  $w$ , decide whether  $w$  belongs to  $L(G)$ .
- If accepted, produce a derivation or tree that explains how  $w$  was generated.
- This is stronger than lexical analysis: nesting and recursion need a stack-like memory.
- Programming language constructs such as balanced parentheses, blocks and expressions are naturally context-free.

### Why a stack is needed

Input tape

Control

Stack

Context-free languages require memory: the stack records expected structure.

```
push: expect )  
match: terminals  
pop: completed nonterminal
```

# Parser responsibilities

## PART 1 • Parsing overview

- Recognize valid token sequences according to the grammar.
- Construct or guide construction of intermediate representation.
- Detect syntax errors as early as possible.
- Recover from errors without flooding the user with cascaded messages.
- Coordinate with symbol-table and semantic-analysis phases through actions or AST nodes.

### Parser responsibilities

Tokens

Parser

AST

Semantic

Parser accepts only token streams that can be generated by the programming-language grammar.

```
if (x < 10) y = y + 1;
```

```
ID ( ID < NUM ) ID = ID + NUM ;
```

# Parsing strategies: big picture

## PART 1 • Parsing overview

- Top-down parsing starts from the start symbol and predicts productions.
- Bottom-up parsing starts from input and reduces handles to the start symbol.
- LL parsers commonly support hand-written predictive parsing.
- LR parsers power many parser generators and handle a larger class of grammars.

Parser family map

| Parser family | Typical strength            | Typical limitation            |
|---------------|-----------------------------|-------------------------------|
| LL(1)         | Simple, hand-written        | Needs grammar transformations |
| SLR           | Small LR tables             | FOLLOW may be too coarse      |
| LALR          | Practical generator default | Some LR(1) distinctions merge |
| CLR/LR(1)     | Most precise                | Large tables                  |

The “right” parser is often the simplest one that accepts the language subset reliably.

# Context-free grammar components

## PART 2 • CFGs and parse trees

- A CFG has terminals, nonterminals, productions and a start symbol.
- Terminals are usually token names returned by the scanner.
- Nonterminals represent syntactic categories: expr, stmt, block, decl.
- Productions describe how a nonterminal can be replaced by a sequence of grammar symbols.

### CFG components

```
E → E + T | T
T → T * F | F
F → ( E ) | id
```

Nonterminals

Terminals

Parsing checks whether the token stream can be derived from the start symbol.

# Example grammar: arithmetic expressions

## PART 2 • CFGs and parse trees

- This classic expression grammar encodes precedence through levels E, T and F.
- E handles addition/subtraction, T handles multiplication/division, F handles operands and parentheses.
- The grammar is clear but left-recursive, so it is natural for LR parsing but not direct LL(1).

```
E → E + T | E - T | T
T → T * F | T / F | F
F → ( E ) | id | num
```

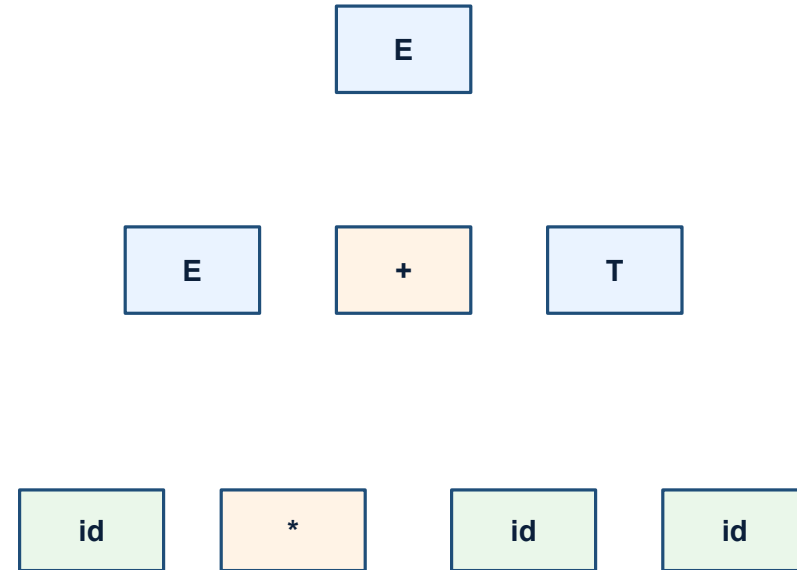
Token stream example:  
id + num \* id

# Derivation: rewriting from start symbol

## PART 2 • CFGs and parse trees

- A derivation is a sequence of production applications.
- Leftmost derivation always rewrites the leftmost nonterminal first.
- Rightmost derivation always rewrites the rightmost nonterminal first.
- Parsing algorithms often correspond to a controlled derivation strategy.

### Derivations explain structure



Parse trees retain full grammar structure; ASTs usually remove syntactic noise.

# Leftmost derivation example

## PART 2 • CFGs and parse trees

- Top-down parsing naturally follows a leftmost derivation.
- Each step predicts how the current leftmost nonterminal expands.
- The final string contains only terminals and should match the input token sequence.

Grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow id$

Leftmost derivation of `id + id`:

$E \Rightarrow T E'$

$\Rightarrow id E'$

$\Rightarrow id + T E'$

$\Rightarrow id + id E'$

$\Rightarrow id + id$

# Rightmost derivation example

## PART 2 • CFGs and parse trees

- Bottom-up parsing in reverse is closely related to rightmost derivation.
- Reductions reconstruct the reverse of a rightmost derivation.
- This is why LR parsing is often described as left-to-right scan, rightmost derivation in reverse.

Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow \text{id}$

Rightmost derivation of  $\text{id} + \text{id}$ :

$E \Rightarrow E + T$

$\Rightarrow E + \text{id}$

$\Rightarrow T + \text{id}$

$\Rightarrow \text{id} + \text{id}$

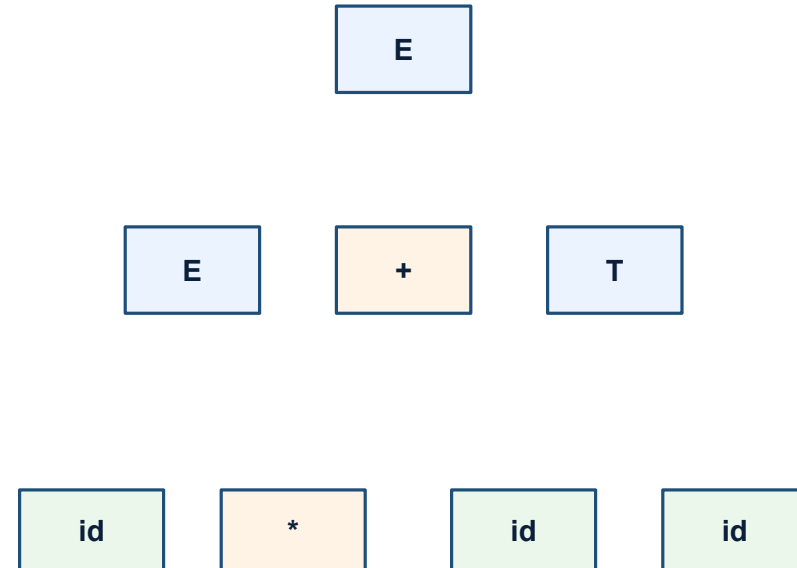
Bottom-up parser reverses this path.

# Parse tree fundamentals

## PART 2 • CFGs and parse trees

- Root is the start symbol.
- Internal nodes are nonterminals.
- Leaves are terminals or  $\epsilon$ .
- Reading leaves left-to-right gives the generated token string.
- Tree shape captures grouping and nesting.

### Parse tree anatomy



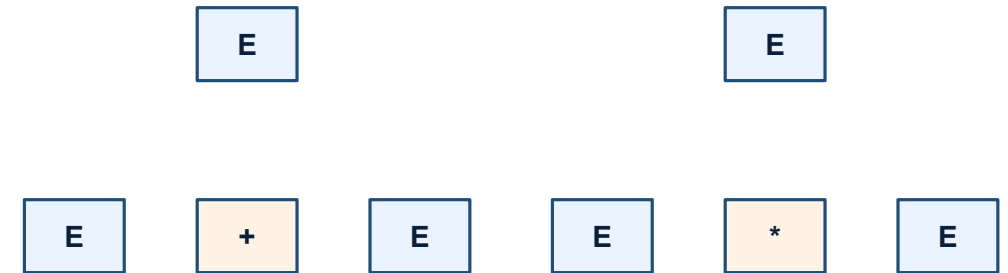
Parse trees retain full grammar structure; ASTs usually remove syntactic noise.

# Ambiguous grammar

## PART 2 • CFGs and parse trees

- A grammar is ambiguous if some string has more than one parse tree.
- Ambiguity causes multiple possible meanings for the same program fragment.
- Common sources: expression operators and dangling else.
- Parser generators may report conflicts when ambiguity reaches table construction.

### Ambiguity: two structures



Same token sequence: id + id \* id  
Different structures  $\Rightarrow$  different meanings.

**Use precedence + associativity**

# Classic ambiguous expression grammar

## PART 2 • CFGs and parse trees

- The grammar is compact but does not encode precedence or associativity.
- The string `id + id * id` can be grouped in two ways.
- A compiler must choose one meaning consistently.

Ambiguous grammar:

```
E → E + E
   | E * E
   | ( E )
   | id
```

Problem string:

`id + id * id`

Meaning 1: `id + (id * id)`

Meaning 2: `(id + id) * id`

# Removing expression ambiguity

## PART 2 • CFGs and parse trees

- Create grammar levels that reflect precedence.
- Higher-precedence operators appear deeper in the grammar.
- Left recursion can encode left associativity for LR parsers.
- Right recursion can encode right associativity when required.

### Unambiguous expression levels

|                       |     |      |
|-----------------------|-----|------|
| $E \rightarrow E + T$ | $ $ | $T$  |
| $T \rightarrow T * F$ | $ $ | $F$  |
| $F \rightarrow ( E )$ | $ $ | $id$ |

Nonterminals

Terminals

Parsing checks whether the token stream can be derived from the start symbol.

# Dangling else ambiguity

## PART 2 • CFGs and parse trees

- Nested if statements can make else binding unclear.
- Most languages bind else to the nearest unmatched if.
- The grammar can be rewritten using matched and unmatched statements.

Ambiguous idea:

```
stmt → if expr then stmt  
      | if expr then stmt else stmt  
      | other
```

Input shape:

```
if E1 then if E2 then S1 else S2
```

Usual rule: else belongs to inner if.

# Grammar quality checklist

## PART 2 • CFGs and parse trees

- Can the grammar generate all intended valid programs?
- Can it generate invalid programs accidentally?
- Is it ambiguous?
- Can the chosen parser family handle it?
- Are grammar transformations preserving the intended language and meaning?

### Before building the parser

- 1 Coverage
- 2 No unintended strings
- 3 No ambiguity
- 4 Parser-compatible
- 5 Good error points

# Top-down parsing intuition

## PART 3 • Top-down parsing and LL(1)

- Begin with the start symbol on a conceptual stack.
- Repeatedly expand the next nonterminal to match the input.
- Use lookahead token to choose the correct production.
- If no production fits, report a syntax error.

### Predictive parsing machine

Input

id + id \$

Stack

\$ E

Action

Use  $M[E, id] = E \rightarrow T E'$

Predictive parsing simulates a leftmost derivation using a stack and one lookahead token.

# Recursive descent parser

## PART 3 • Top-down parsing and LL(1)

- One procedure is written for each nonterminal.
- Each procedure consumes tokens that match its grammar fragment.
- Simple grammars can be parsed with nested if/switch statements.
- Backtracking is possible but avoided in efficient compiler parsers.

### Procedure per nonterminal

Use the example on the left to trace parser behavior.

Input

Stack

Action

# Recursive descent code pattern

## PART 3 • Top-down parsing and LL(1)

- The parser looks at the next token and selects a production.
- `match(t)` checks and consumes a terminal.
- For LL(1), one token of lookahead is enough to choose without backtracking.

```
void E() {
    T();
    Eprime();
}

void Eprime() {
    if (lookahead == PLUS) {
        match(PLUS);
        T();
        Eprime();
    } else {
        // ε production
    }
}
```

# Problem: left recursion

## PART 3 • Top-down parsing and LL(1)

- A top-down parser that tries  $A \rightarrow A \alpha$  first can recurse forever.
- Direct left recursion:  $A \rightarrow A \alpha \mid \beta$ .
- Indirect left recursion:  $A \Rightarrow^+ A \alpha$  through multiple nonterminals.
- LL parsing requires removing left recursion.

**Left recursion causes infinite descent**

$E \rightarrow E + T \mid T$

$E()$  calls  $E()$  before consuming input.

Nonterminals

Terminals

Parsing checks whether the token stream can be derived from the start symbol.

# Eliminating direct left recursion

## PART 3 • Top-down parsing and LL(1)

- General form:  $A \rightarrow A \alpha \mid \beta$ .
- Replace by  $A \rightarrow \beta A'$  and  $A' \rightarrow \alpha A' \mid \epsilon$ .
- This preserves the language but changes parse tree shape.
- Semantic actions may need adjustment after transformation.

Original:

$A \rightarrow A \alpha \mid \beta$

Transformed:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Expression example:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

# Problem: common prefixes

## PART 3 • Top-down parsing and LL(1)

- When two productions start similarly, a predictive parser cannot choose early.
- Left factoring delays the decision until enough input has been seen.
- This is a grammar transformation for top-down parsing.

### Common prefix before factoring

```
stmt → if expr then stmt else stmt  
      | if expr then stmt
```

Nonterminals

Terminals

Parsing checks whether the token stream can be derived from the start symbol.

# Left factoring transformation

## PART 3 • Top-down parsing and LL(1)

- General form:  $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ .
- Factor the common prefix  $\alpha$  into a separate production.
- Choice between  $\beta_1$  and  $\beta_2$  happens later.

Original:

$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$

Factored:

$A \rightarrow \alpha A'$

$A' \rightarrow \beta_1 \mid \beta_2$

If example:

$\text{stmt} \rightarrow \text{if expr then stmt opt\_else}$

$\text{opt\_else} \rightarrow \text{else stmt} \mid \epsilon$

# FIRST set: meaning

## PART 3 • Top-down parsing and LL(1)

- $\text{FIRST}(\alpha)$  is the set of terminals that can begin strings derived from  $\alpha$ .
- If  $\alpha$  can derive  $\epsilon$ , then  $\epsilon$  belongs to  $\text{FIRST}(\alpha)$ .
- For a terminal  $a$ ,  $\text{FIRST}(a) = \{a\}$ .
- For a sequence  $X Y$ ,  $\text{FIRST}(XY)$  includes  $\text{FIRST}(X)$  minus  $\epsilon$ ; if  $X$  is nullable, include  $\text{FIRST}(Y)$ .

### FIRST set intuition

| Symbol | FIRST             | FOLLOW       |
|--------|-------------------|--------------|
| E      | { id, ( }         | { \$, ) }    |
| E'     | { +, $\epsilon$ } | { \$, ) }    |
| T      | { id, ( }         | { +, \$, ) } |
| T'     | { *, $\epsilon$ } | { +, \$, ) } |

FIRST predicts what can begin; FOLLOW tells what may legally come after.

# FIRST set algorithm sketch

## PART 3 • Top-down parsing and LL(1)

- Initialize FIRST(a) for terminals and FIRST( $\epsilon$ ).
- Iteratively inspect productions until no set changes.
- For  $A \rightarrow X_1 X_2 \dots X_k$ , add FIRST( $X_i$ ) in order while previous symbols are nullable.

```
repeat
  for each production  $A \rightarrow X_1 X_2 \dots X_k$ :
    add FIRST( $X_1$ ) -  $\{\epsilon\}$  to FIRST( $A$ )
    if  $X_1$  nullable, add FIRST( $X_2$ ) -  $\{\epsilon\}$ 
    ...
    if all  $X_i$  nullable, add  $\epsilon$  to FIRST( $A$ )
until no FIRST set changes
```

# FOLLOW set: meaning

## PART 3 • Top-down parsing and LL(1)

- FOLLOW(A) is the set of terminals that may appear immediately after A in some sentential form.
- The end marker \$ is in FOLLOW(start symbol).
- FOLLOW sets decide when an  $\epsilon$  production should be used.
- FOLLOW also supports SLR reductions in bottom-up parsing.

### FOLLOW set intuition

| Symbol | FIRST             | FOLLOW       |
|--------|-------------------|--------------|
| E      | { id, ( }         | { \$, ) }    |
| E'     | { +, $\epsilon$ } | { \$, ) }    |
| T      | { id, ( }         | { +, \$, ) } |
| T'     | { *, $\epsilon$ } | { +, \$, ) } |

FIRST predicts what can begin; FOLLOW tells what may legally come after.

# FOLLOW set algorithm sketch

## PART 3 • Top-down parsing and LL(1)

- Initialize FOLLOW(S) with \$.
- For  $A \rightarrow \alpha B \beta$ , add FIRST( $\beta$ ) minus  $\epsilon$  to FOLLOW(B).
- If  $\beta$  is nullable or absent, add FOLLOW(A) to FOLLOW(B).
- Iterate until no set changes.

```
FOLLOW(S) = { $ }
repeat
  for each production  $A \rightarrow \alpha B \beta$ :
    add FIRST( $\beta$ ) - { $\epsilon$ } to FOLLOW(B)
    if  $\beta \Rightarrow^* \epsilon$ :
      add FOLLOW(A) to FOLLOW(B)
until no FOLLOW set changes
```

# LL(1) condition

## PART 3 • Top-down parsing and LL(1)

- For each nonterminal A, alternatives must be distinguishable by one lookahead token.
- FIRST sets of alternatives should not overlap.
- If an alternative can produce  $\epsilon$ , FIRST(other alternatives) must not overlap FOLLOW(A).
- Equivalent table test: no cell should contain more than one production.

LL(1) table has unique entries

| M[A,a] | id                  | +                         | *                     | \$                        |
|--------|---------------------|---------------------------|-----------------------|---------------------------|
| E      | $E \rightarrow TE'$ |                           |                       |                           |
| E'     |                     | $E' \rightarrow +TE'$     |                       | $E' \rightarrow \epsilon$ |
| T      | $T \rightarrow FT'$ |                           |                       |                           |
| T'     |                     | $T' \rightarrow \epsilon$ | $T' \rightarrow *FT'$ | $T' \rightarrow \epsilon$ |

LL(1) means no table cell needs two productions.

# Constructing an LL(1) parsing table

## PART 3 • Top-down parsing and LL(1)

- For each production  $A \rightarrow \alpha$ , place it in columns for terminals in  $\text{FIRST}(\alpha)$ .
- If  $\epsilon$  is in  $\text{FIRST}(\alpha)$ , also place it in columns from  $\text{FOLLOW}(A)$ .
- Blank cells represent syntax errors.
- Multiple entries in one cell mean grammar is not LL(1).

```
for each production  $A \rightarrow \alpha$ :  
  for each terminal  $a$  in  $\text{FIRST}(\alpha) - \{\epsilon\}$ :  
     $M[A, a] = A \rightarrow \alpha$   
  if  $\epsilon$  in  $\text{FIRST}(\alpha)$ :  
    for each  $b$  in  $\text{FOLLOW}(A)$ :  
       $M[A, b] = A \rightarrow \alpha$ 
```

# LL(1) table example

## PART 3 • Top-down parsing and LL(1)

- Rows are nonterminals; columns are lookahead terminals.
- A stack top nonterminal A and input token a select  $M[A,a]$ .
- Terminal on stack must match the input token.
- End marker \$ confirms successful completion.

Table-driven predictive parsing

| $M[A,a]$ | id                  | +                         | *                     | \$                        |
|----------|---------------------|---------------------------|-----------------------|---------------------------|
| E        | $E \rightarrow TE'$ |                           |                       |                           |
| E'       |                     | $E' \rightarrow +TE'$     |                       | $E' \rightarrow \epsilon$ |
| T        | $T \rightarrow FT'$ |                           |                       |                           |
| T'       |                     | $T' \rightarrow \epsilon$ | $T' \rightarrow *FT'$ | $T' \rightarrow \epsilon$ |

LL(1) means no table cell needs two productions.

# Predictive parser algorithm

## PART 3 • Top-down parsing and LL(1)

- Initialize stack with \$ and start symbol.
- If top is terminal and equals lookahead, pop and advance.
- If top is nonterminal, use parsing table to push production body in reverse.
- If top and lookahead are both \$, accept.

### Stack + input + table

Input

id + id \$

Stack

\$ E

Action

Use  $M[E, id] = E \rightarrow T E'$

Predictive parsing simulates a leftmost derivation using a stack and one lookahead token.

# LL(1) trace: id + id

## PART 3 • Top-down parsing and LL(1)

- Trace format: stack top is shown at the right.
- A production replaces a nonterminal on the stack.
- Terminals are matched and consumed from input.

| Step | Stack     | Input      | Action                    |
|------|-----------|------------|---------------------------|
| 1    | \$ E      | id + id \$ | $E \rightarrow T E'$      |
| 2    | \$ E' T   | id + id \$ | $T \rightarrow id$        |
| 3    | \$ E' id  | id + id \$ | match id                  |
| 4    | \$ E'     | + id \$    | $E' \rightarrow + T E'$   |
| 5    | \$ E' T + | + id \$    | match +                   |
| 6    | \$ E' T   | id \$      | $T \rightarrow id$        |
| 7    | \$ E' id  | id \$      | match id                  |
| 8    | \$ E'     | \$         | $E' \rightarrow \epsilon$ |
| 9    | \$        | \$         | accept                    |

# Limits of LL(1)

## PART 3 • Top-down parsing and LL(1)

- Some natural grammars are not LL(1) even after simple transformations.
- One token lookahead may be insufficient for certain constructs.
- Left recursion must be removed, which may complicate semantic actions.
- Bottom-up LR parsing accepts a larger class of grammars.

### Why move to LR?

| Parser family | Typical strength            | Typical limitation            |
|---------------|-----------------------------|-------------------------------|
| LL(1)         | Simple, hand-written        | Needs grammar transformations |
| SLR           | Small LR tables             | FOLLOW may be too coarse      |
| LALR          | Practical generator default | Some LR(1) distinctions merge |
| CLR/LR(1)     | Most precise                | Large tables                  |

The “right” parser is often the simplest one that accepts the language subset reliably.

# Bottom-up parsing intuition

## PART 4 • Bottom-up parsing and LR family

- Start with the input tokens and try to reconstruct the start symbol.
- Shift tokens onto a stack until a right-hand side of a production is recognized.
- Reduce that right-hand side to its left-hand nonterminal.
- Accept when only the start symbol and end marker remain.

### Shift-reduce cycle

Input

id + id \$

Stack

\$ E

Action

Use  $M[E, id] = E \rightarrow T E'$

Predictive parsing simulates a leftmost derivation using a stack and one lookahead token.

# Shift-reduce trace: id + id

## PART 4 • Bottom-up parsing and LR family

- This trace uses  $E \rightarrow E + T \mid T$  and  $T \rightarrow \text{id}$ .
- Reductions are the reverse of a rightmost derivation.
- The stack contains grammar symbols and later also parser states in LR implementations.

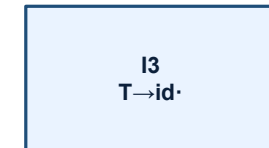
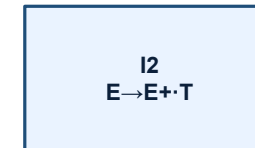
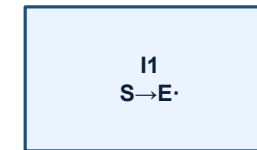
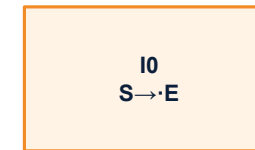
| Step | Stack     | Input      | Action                           |
|------|-----------|------------|----------------------------------|
| 1    | \$        | id + id \$ | shift id                         |
| 2    | \$ id     | + id \$    | reduce $T \rightarrow \text{id}$ |
| 3    | \$ T      | + id \$    | reduce $E \rightarrow T$         |
| 4    | \$ E      | + id \$    | shift +                          |
| 5    | \$ E +    | id \$      | shift id                         |
| 6    | \$ E + id | \$         | reduce $T \rightarrow \text{id}$ |
| 7    | \$ E + T  | \$         | reduce $E \rightarrow E + T$     |
| 8    | \$ E      | \$         | accept                           |

# Handle: what to reduce

## PART 4 • Bottom-up parsing and LR family

- A handle is a substring matching a production body whose reduction is valid in a rightmost derivation.
- Reducing a wrong substring can destroy the parse.
- LR automata track enough context to locate handles efficiently.
- This is the key difference between naive bottom-up parsing and LR parsing.

### Automata locate handles



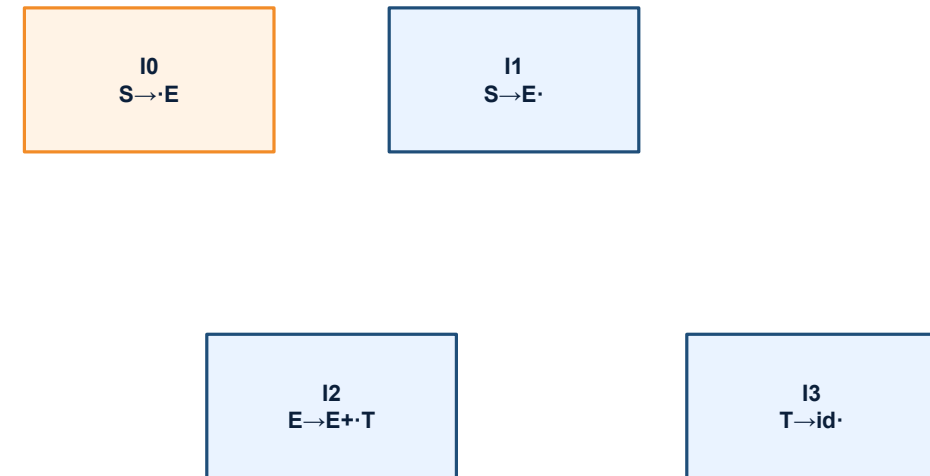
States summarize what production bodies may be in progress.

# Viable prefixes

## PART 4 • Bottom-up parsing and LR family

- A viable prefix is a prefix of a right-sentential form that does not extend beyond a handle.
- The LR stack always contains a viable prefix.
- The set of viable prefixes is regular, so a finite automaton can recognize it.
- This explains why LR parsing uses DFA states plus a stack.

### DFA for viable prefixes



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# LR parser architecture

## PART 4 • Bottom-up parsing and LR family

- The stack stores states, often interleaved with grammar symbols conceptually.
- ACTION[state, token] chooses shift, reduce, accept or error.
- GOTO[state, nonterminal] gives the next state after a reduction.
- The driver algorithm is generic; tables encode the grammar.

### ACTION and GOTO tables

Use the example on the left to trace parser behavior.

Input

Stack

Action

# LR parsing driver algorithm

## PART 4 • Bottom-up parsing and LR family

- Let  $s$  be the current state on top of stack and  $a$  the lookahead token.
- Shift pushes token and next state; reduce pops the production body length.
- After reducing  $A \rightarrow \beta$ , GOTO determines the state after pushing  $A$ .

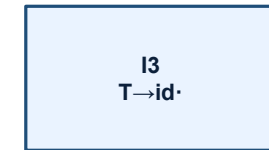
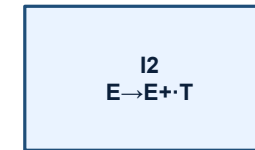
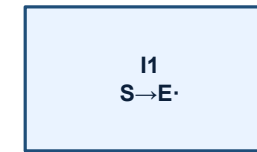
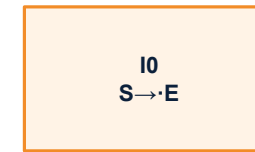
```
push state 0
a = nextToken()
loop:
  s = topState()
  action = ACTION[s, a]
  if action = shift t:
    push a; push state t; a = nextToken()
  else if action = reduce  $A \rightarrow \beta$ :
    pop  $2 * |\beta|$  entries
    t = topState()
    push A; push GOTO[t, A]
  else if action = accept:
    success
  else:
    syntax_error()
```

# LR items: marker position

## PART 4 • Bottom-up parsing and LR family

- An LR(0) item is a production with a dot marking progress.
- $A \rightarrow \alpha \cdot \beta$  means  $\alpha$  has been recognized;  $\beta$  remains expected.
- $A \rightarrow \alpha \cdot$  means the production body has been completely recognized.
- Items are grouped into states of the LR automaton.

### LR(0) item states



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# Item examples

## PART 4 • Bottom-up parsing and LR family

- The same production creates multiple items depending on dot position.
- Dot before a terminal suggests a possible shift.
- Dot at the end suggests a possible reduction.
- Dot before a nonterminal triggers closure expansion.

Production:

$E \rightarrow E + T$

Items:

$E \rightarrow \cdot E + T$

$E \rightarrow E \cdot + T$

$E \rightarrow E + \cdot T$

$E \rightarrow E + T \cdot$

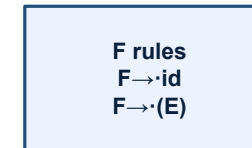
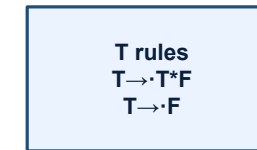
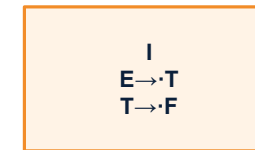
Meaning: dot shows how much of RHS is recognized.

# Closure operation

## PART 4 • Bottom-up parsing and LR family

- If an item has a dot before nonterminal B, the parser may need to parse a B next.
- Closure adds items for all productions of B with the dot at the beginning.
- Repeat until no new items can be added.
- Closure makes implicit expectations explicit inside a state.

### CLOSURE expands expectations



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# Closure algorithm

## PART 4 • Bottom-up parsing and LR family

- Input: a set of items  $I$ .
- Whenever dot is before a nonterminal, add its productions.
- The result is one LR automaton state.

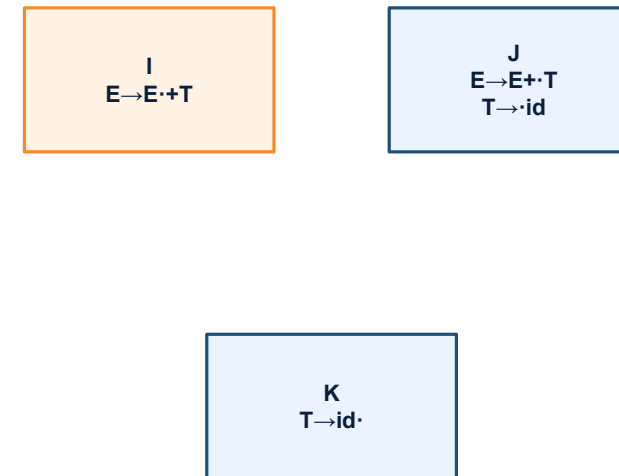
```
CLOSURE(I):  
  repeat  
    for each item  $A \rightarrow \alpha \cdot B \beta$  in  $I$ :  
      for each production  $B \rightarrow \gamma$ :  
        add item  $B \rightarrow \cdot \gamma$  to  $I$   
  until no more items can be added  
  return  $I$ 
```

# GOTO operation

## PART 4 • Bottom-up parsing and LR family

- $\text{GOTO}(I, X)$  moves the dot over grammar symbol  $X$  in all items where  $X$  appears after the dot.
- Then closure is applied to the moved items.
- GOTO edges become transitions in the LR automaton.
- Terminals lead to shift actions; nonterminals lead to GOTO table entries.

### GOTO as dot movement



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# GOTO algorithm

## PART 4 • Bottom-up parsing and LR family

- Move the dot over X.
- Discard items where X is not immediately after the dot.
- Apply closure to expose new expectations.

```
GOTO(I, X):  
  J =  $\emptyset$   
  for each item  $A \rightarrow \alpha \cdot X \beta$  in I:  
    add  $A \rightarrow \alpha X \cdot \beta$  to J  
  return CLOSURE(J)
```

# Augmented grammar

## PART 4 • Bottom-up parsing and LR family

- LR construction begins with an augmented start production  $S' \rightarrow S$ .
- Accept occurs when  $S' \rightarrow S \cdot$  is seen with end marker \$.
- Augmentation gives the parser a unique completion condition.
- The new start symbol is not part of the source language.

### Augmented start production

```
S' → S
S  → E
E  → E + T | T
T  → id
```

Nonterminals

Terminals

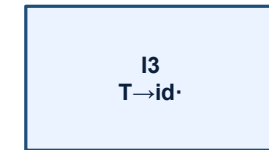
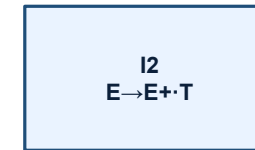
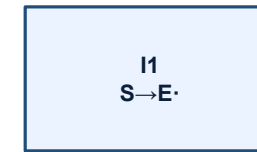
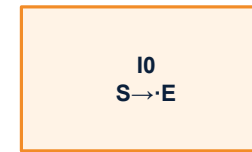
Parsing checks whether the token stream can be derived from the start symbol.

# Constructing canonical LR(0) item sets

## PART 4 • Bottom-up parsing and LR family

- Start with  $\text{CLOSURE}(\{S' \rightarrow \cdot S\})$ .
- For every state  $I$  and grammar symbol  $X$ , compute  $\text{GOTO}(I, X)$ .
- If the resulting set is new, add it as a new state.
- Continue until no new states appear.

### Canonical collection as a DFA



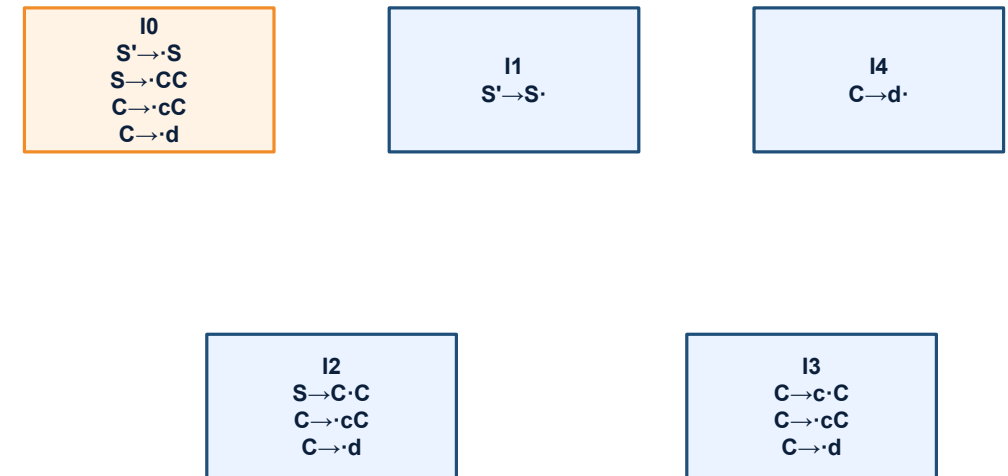
LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# LR(0) automaton: small grammar

## PART 4 • Bottom-up parsing and LR family

- Consider  $S' \rightarrow S$ ,  $S \rightarrow C C$ ,  $C \rightarrow c C \mid d$ .
- This grammar is often used to show item-set construction.
- Each item set becomes a DFA state.
- Transitions are labeled by terminals or nonterminals.

### LR(0) DFA sketch



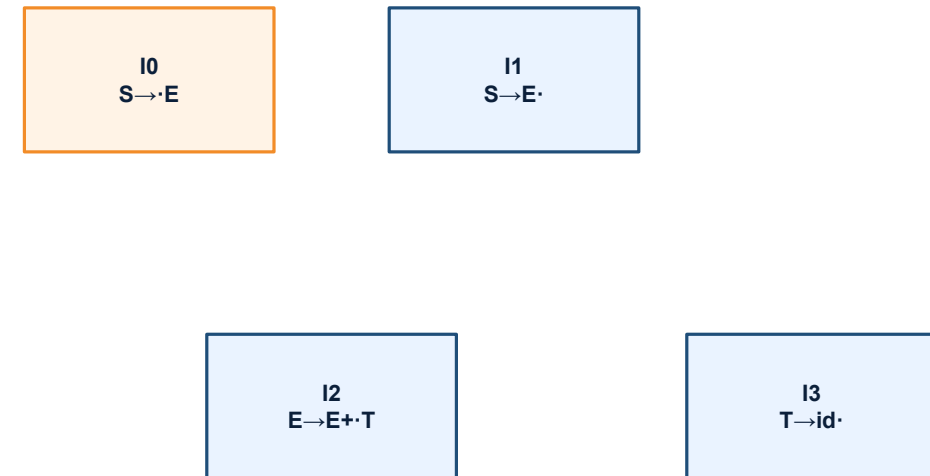
LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# LR(0) table construction

## PART 4 • Bottom-up parsing and LR family

- If item  $A \rightarrow \alpha \cdot a \beta$  exists, set  $\text{ACTION}[\text{state}, a] = \text{shift GOTO}(\text{state}, a)$ .
- If item  $A \rightarrow \alpha \cdot$  exists, set reduce  $A \rightarrow \alpha$  on all terminal columns in LR(0).
- If  $S' \rightarrow S \cdot$  exists, set  $\text{ACTION}[\text{state}, \$] = \text{accept}$ .
- Conflicts mean grammar is not LR(0).

### From items to ACTION/GOTO



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# SLR parsing idea

## PART 4 • Bottom-up parsing and LR family

- SLR uses the LR(0) automaton but restricts reductions by FOLLOW(left-hand side).
- For  $A \rightarrow \alpha \cdot$ , reduce only when lookahead is in FOLLOW(A).
- This is simple and often works for teaching grammars.
- It can still be too coarse for some grammars because FOLLOW ignores LR state context.

### FOLLOW controls reductions

| Symbol | FIRST             | FOLLOW       |
|--------|-------------------|--------------|
| E      | { id, ( }         | { \$, ) }    |
| E'     | { +, $\epsilon$ } | { \$, ) }    |
| T      | { id, ( }         | { +, \$, ) } |
| T'     | { *, $\epsilon$ } | { +, \$, ) } |

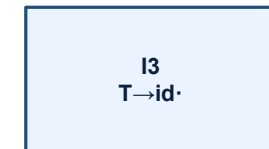
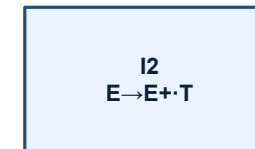
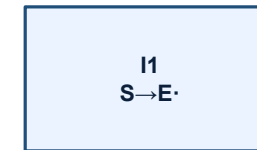
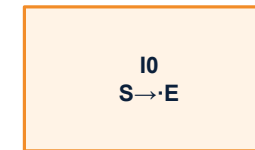
FIRST predicts what can begin; FOLLOW tells what may legally come after.

# LR(1) items

## PART 4 • Bottom-up parsing and LR family

- An LR(1) item adds one-token lookahead to an LR(0) item.
- Form:  $[A \rightarrow \alpha \cdot \beta, a]$ .
- Lookahead  $a$  tells when the completed production may be reduced.
- LR(1) uses local context rather than only FOLLOW(A).

LR(1) item = LR(0) item + lookahead



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# LR(1) closure intuition

## PART 4 • Bottom-up parsing and LR family

- When dot is before B in  $[A \rightarrow \alpha \cdot B \beta, a]$ , add B productions.
- The lookahead for B items comes from  $\text{FIRST}(\beta a)$ .
- This propagates context more precisely than SLR.

LR(1) closure rule:

```
[A → α · B β, a]
For every production B → γ
For every b in FIRST(β a):
    add [B → · γ, b]
```

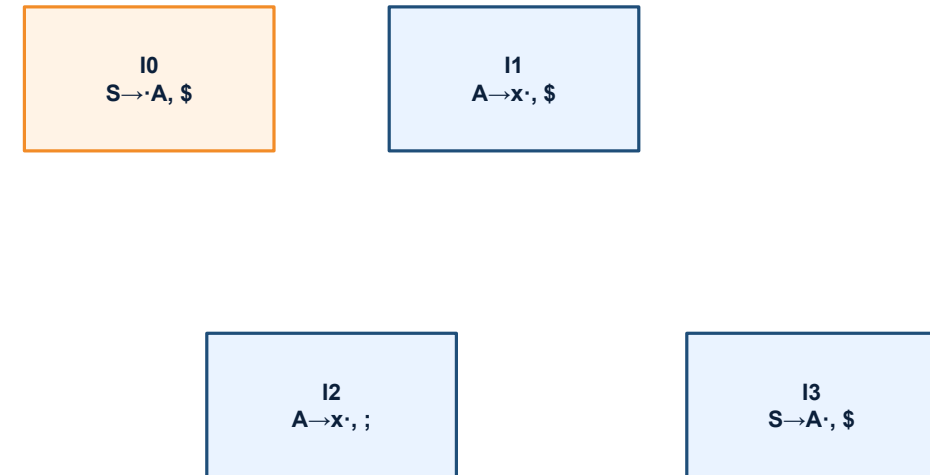
Lookahead belongs to the reduction context,  
not to the immediate shift symbol.

# Canonical LR(1) strength

## PART 4 • Bottom-up parsing and LR family

- Canonical LR(1), or CLR, is the most precise common LR table construction.
- It can parse many grammars that SLR cannot.
- The cost is more states because lookahead differences split item sets.
- The algorithm is elegant but table size can be large.

### CLR has more context states



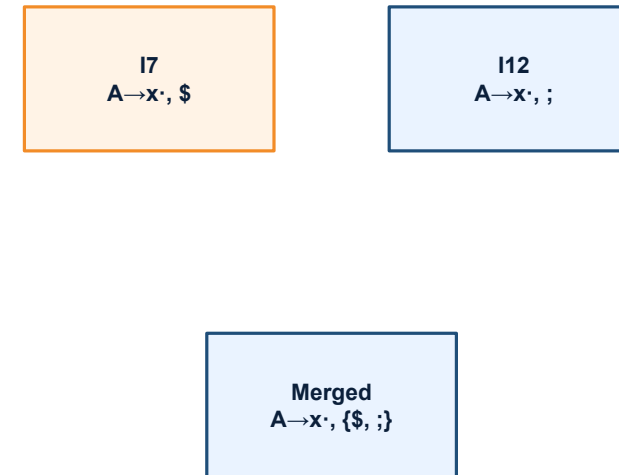
LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# LALR parsing idea

## PART 4 • Bottom-up parsing and LR family

- LALR merges LR(1) states that have the same LR(0) core.
- Lookahead sets are combined after merging.
- This usually gives tables similar in size to SLR but with much more power.
- Yacc-style generators traditionally use LALR(1).

### Merging same cores



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# SLR vs CLR vs LALR

## PART 4 • Bottom-up parsing and LR family

- All are LR-style deterministic bottom-up parsers.
- They differ mainly in how much lookahead context is used for reductions.
- In practice, LALR is a strong balance for compiler construction courses and tools.

| Method    | States       | Lookahead precision | Typical use                   |
|-----------|--------------|---------------------|-------------------------------|
| LR(0)     | Small        | None                | Teaching baseline             |
| SLR(1)    | Small        | FOLLOW sets         | Simple grammars               |
| LALR(1)   | Small-medium | Merged LR(1)        | Yacc/Bison-style tools        |
| CLR/LR(1) | Large        | Full LR(1)          | Theory and precise generators |

# Shift-reduce conflict

## PART 5 • Conflicts, ambiguity and precedence

- Occurs when a table cell has both shift and reduce actions.
- Common with expression grammars and dangling else.
- Sometimes resolved by grammar rewriting.
- Parser generators also support precedence and associativity declarations.

### Shift vs reduce decision

State I

$E \rightarrow E \cdot + T$   
 $E \rightarrow E \cdot$

shift +

reduce E

A conflict means the parser table cannot decide one unique action for a state/lookahead pair.

# Reduce-reduce conflict

## PART 5 • Conflicts, ambiguity and precedence

- Occurs when two completed items both want to reduce on the same lookahead.
- Usually indicates a serious grammar design problem.
- The parser cannot know which nonterminal the same input phrase should become.
- Reduce-reduce conflicts should normally be removed, not patched.

### Two possible reductions

State I

$E \rightarrow E \cdot + T$   
 $E \rightarrow E \cdot$

shift +

reduce E

A conflict means the parser table cannot decide one unique action for a state/lookahead pair.

# Conflict example: expression grammar

## PART 5 • Conflicts, ambiguity and precedence

- The compact grammar  $E \rightarrow E + E \mid E * E \mid \text{id}$  is ambiguous.
- At  $E + E \cdot$  with lookahead  $*$ , should parser reduce or shift?
- Language convention says shift  $*$  because multiplication has higher precedence.

Ambiguous grammar:

```
E → E + E
   | E * E
   | id
```

Input prefix seen:

$\text{id} + \text{id} \cdot * \text{id}$

Decision:

reduce  $\text{id} + \text{id}$  first?

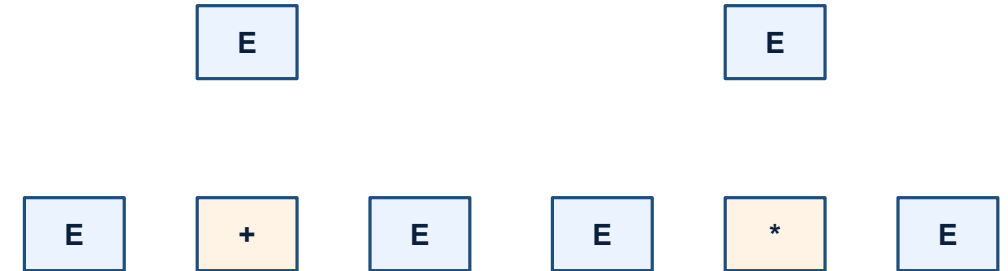
or shift  $*$  and parse  $\text{id} * \text{id}$  first?

# Operator precedence as conflict policy

## PART 5 • Conflicts, ambiguity and precedence

- Precedence assigns priority levels to operators.
- Associativity resolves repeated operators at the same precedence level.
- Left associative:  $a-b-c$  parses as  $(a-b)-c$ .
- Right associative:  $a=b=c$  parses as  $a=(b=c)$ .

### Precedence shapes parse tree



Same token sequence:  $id + id * id$   
Different structures  $\Rightarrow$  different meanings.

**Use precedence + associativity**

# Grammar-based precedence

## PART 5 • Conflicts, ambiguity and precedence

- A grammar can encode precedence without tool-specific declarations.
- Separate nonterminals are used for each precedence level.
- This is explicit and portable, but often longer.

```
expr  → expr + term | expr - term | term
term  → term * factor | term / factor | factor
factor → ( expr ) | ID | NUM
```

Higher precedence is deeper in the grammar.

# Declaration-based precedence in Yacc/Bison

## PART 5 • Conflicts, ambiguity and precedence

- Parser generators can resolve expression conflicts using declarations.
- The grammar stays compact while precedence rules guide table conflict resolution.
- This is convenient but students should still understand the underlying ambiguity.

```
%left '+' '-'  
%left '*' '/'  
%right UMINUS  
  
%%  
expr : expr '+' expr  
      | expr '*' expr  
      | '-' expr %prec UMINUS  
      | '(' expr ')'  
      | ID  
      ;
```

# Dangling else conflict

## PART 5 • Conflicts, ambiguity and precedence

- At an else token, parser may reduce an if-without-else or shift else.
- Default shift attaches else to the nearest unmatched if.
- A grammar rewrite can also enforce the policy explicitly.
- Understanding this conflict helps students interpret parser-generator warnings.

### Dangling else: shift is conventional

State I

$E \rightarrow E \cdot + T$   
 $E \rightarrow E \cdot$

shift +

reduce E

A conflict means the parser table cannot decide one unique action for a state/lookahead pair.

# Matched/unmatched solution

## PART 5 • Conflicts, ambiguity and precedence

- Separate statements that are complete from those waiting for an else.
- This removes ambiguity by construction.
- The grammar is longer but makes the language rule explicit.

```
stmt      → matched | unmatched
matched   → if expr then matched else matched
           | other
unmatched → if expr then stmt
           | if expr then matched else unmatched
```

Policy: else binds to nearest unmatched if.

# Reading parser-generator conflicts

## PART 5 • Conflicts, ambiguity and precedence

- Do not ignore conflicts without understanding the cause.
- Check the state that contains the conflict and the competing items.
- Identify the input pattern that reaches the state.
- Decide whether to rewrite grammar or add precedence declarations.

### Conflict debugging

- 1 Locate conflict state
- 2 Inspect items
- 3 Find sample input
- 4 Classify conflict
- 5 Fix grammar or policy

# Conflict resolution strategy

## PART 5 • Conflicts, ambiguity and precedence

- Prefer unambiguous grammar for core language structure.
- Use precedence declarations for standard expression operators.
- Avoid reduce-reduce conflicts by redesigning grammar categories.
- Keep small test inputs for each grammar rule and each conflict case.

### Safe resolution plan

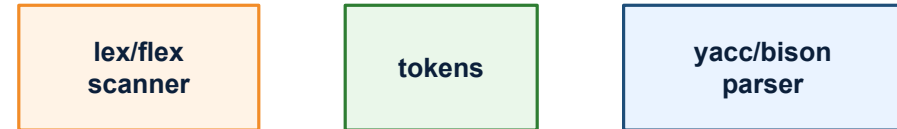
- 1 Rewrite when meaning matters
- 2 Use precedence for operators
- 3 Avoid reduce-reduce
- 4 Retest grammar
- 5 Document choices

# What Yacc/Bison generates

## PART 6 • Yacc/Bison parser generation

- Input: grammar rules, token declarations and semantic actions.
- Output: a parser function, usually `yyparse()`.
- Parser calls `yylex()` to request the next token from scanner.
- The generated parser is commonly LALR(1), with modern variants in newer tools.

### Generated parser workflow



```
%token ID NUM
%left '+'
%%
expr: expr '+' term { ... }
    | term ;
```

Yacc describes grammar rules and actions; Lex/Flex provides `yylex()`.

# Structure of a Yacc/Bison file

## PART 6 • Yacc/Bison parser generation

- Definitions section declares tokens, types and precedence.
- Rules section gives grammar productions and actions.
- User code section contains helper functions and main program.

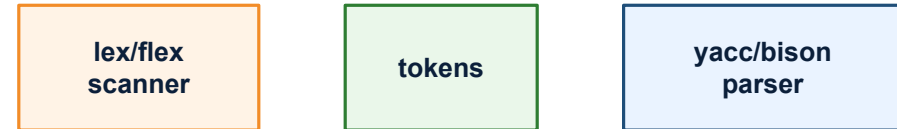
```
%{  
    /* C declarations */  
%}  
  
%token ID NUM  
%left '+' '-'  
%left '*' '/'  
  
%%  
program : stmt_list ;  
stmt_list : stmt_list stmt | /* empty */ ;  
stmt : ID '=' expr ';' ;  
expr : expr '+' term | term ;  
%%  
int main() { return yyparse(); }
```

# Token interface between Flex and Bison

## PART 6 • Yacc/Bison parser generation

- Flex scanner recognizes lexemes and returns token codes.
- Bison parser consumes token codes and semantic values.
- Both files must agree on token names.
- The parser does not care how the scanner found the token.

**yylex() returns tokens**



```
%token ID NUM
%left '+'
%%
expr: expr '+' term { ... }
    | term ;
```

Yacc describes grammar rules and actions; Lex/Flex provides yylex().

# Flex scanner fragment for parser

## PART 6 • Yacc/Bison parser generation

- Identifiers and numbers return token names declared in Bison.
- Operators can be returned as character literals.
- Whitespace is skipped by the scanner.
- Semantic values can be assigned through `yylval`.

```
[0-9]+      { yylval.ival = atoi(yytext); return NUM; }
[a-zA-Z_][a-zA-Z0-9]* { return ID; }
[
]+         { /* skip whitespace */ }
"+"        { return '+'; }
"*"        { return '*'; }
"("        { return '('; }
")"        { return ')'; }
.          { return yytext[0]; }
```

# Bison expression grammar

## PART 6 • Yacc/Bison parser generation

- This grammar uses explicit precedence by grammar levels.
- Actions can compute values or construct AST nodes.
- \$\$ is the value of the left-hand side; \$1, \$2, ... are RHS values.

```
expr : expr '+' term { $$ = makeAdd($1, $3); }  
    | expr '-' term { $$ = makeSub($1, $3); }  
    | term          { $$ = $1; }  
    ;  
  
term : term '*' factor { $$ = makeMul($1, $3); }  
    | factor          { $$ = $1; }  
    ;  
  
factor : '(' expr ')' { $$ = $2; }  
       | NUM          { $$ = makeNum($1); }  
       | ID           { $$ = makeId($1); }  
       ;
```

# Using semantic value types

## PART 6 • Yacc/Bison parser generation

- Bison can store different semantic value types in a union.
- Tokens and nonterminals can be assigned value fields.
- Type discipline helps avoid confusion in semantic actions.

```
%union {  
    int ival;  
    char* sval;  
    struct AST* node;  
}  
  
%token <ival> NUM  
%token <sval> ID  
%type <node> expr term factor stmt  
  
expr : expr '+' term { $$ = makeAdd($1, $3); } ;
```

# Unary minus with %prec

## PART 6 • Yacc/Bison parser generation

- The token - may be binary or unary depending on context.
- %prec UMINUS assigns a special precedence to the unary production.
- This avoids ambiguous conflict behavior.

```
%left '+' '-'  
%left '*' '/'  
%right UMINUS  
  
%%  
expr : expr '-' expr  
      | expr '*' expr  
      | '-' expr %prec UMINUS  
      | '(' expr ')'  
      | NUM  
      ;
```

# Parser generation build flow

## PART 6 • Yacc/Bison parser generation

- bison/yacc processes grammar file and generates parser source and header.
- flex processes scanner specification and generates scanner source.
- C compiler links generated sources with helper code.
- Running the executable starts yyparse(), which repeatedly calls yylex().

### Build flow

lex/flex  
scanner

tokens

yacc/bison  
parser

```
%token ID NUM
%left '+'
%%
expr: expr '+' term { ... }
    | term ;
```

Yacc describes grammar rules and actions; Lex/Flex provides yylex().

# Typical lab commands

## PART 6 • Yacc/Bison parser generation

- Use -d to generate token definitions for the scanner.
- Compile parser, scanner and support code together.
- Command names vary by platform and installation.

```
bison -d parser.y          # produces parser.tab.c and parser.tab.h
flex scanner.l             # produces lex.yy.c
gcc parser.tab.c lex.yy.c -lfl -o parser
./parser < input.txt
```

Legacy variants:  
yacc -d parser.y  
lex scanner.l

# Common Yacc/Bison mistakes

## PART 6 • Yacc/Bison parser generation

- Scanner returns token names not declared in the parser.
- Grammar uses character literals inconsistently with scanner returns.
- Semantic value types are missing or mismatched.
- Empty productions are forgotten for optional/list constructs.
- Conflicts are ignored instead of being analyzed.

### Practical lab checklist

- 1 Match token names
- 2 Return semantic values
- 3 Declare precedence
- 4 Test empty cases
- 5 Read conflicts

# Parse tree vs AST

## PART 7 • Syntax-directed translation

- Parse tree includes every grammar symbol and punctuation token.
- AST removes syntactic scaffolding and focuses on operations, operands and language constructs.
- AST is much easier for type checking and code generation.
- Parser actions often build AST nodes as reductions complete.

### Parse tree to AST

Parse Tree

Prune punctuation

AST

```
Parse tree: E → E + T → id + id
AST:      +(id, id)
```

AST is the usual interface from parsing to semantic analysis and code generation.

# AST for expression $a + b * c$

## PART 7 • Syntax-directed translation

- The AST encodes precedence through tree shape.
- The multiplication node becomes child of addition.
- Parentheses usually affect shape but are not kept as separate AST nodes.

Source expression:

$a + b * c$

AST shape:



Functional form:  $+(a, *(b, c))$

# Syntax-directed translation

## PART 7 • Syntax-directed translation

- Attach rules or actions to grammar productions.
- Actions compute attributes or build intermediate representations.
- In bottom-up parsing, actions naturally execute at reduction time.
- In top-down parsing, actions must be placed carefully to respect needed values.

### Actions attach meaning to syntax

Parse Tree

Prune punctuation

AST

```
Parse tree:  E → E + T → id + id
AST:        +(id, id)
```

AST is the usual interface from parsing to semantic analysis and code generation.

# S-attributed evaluation example

## PART 7 • Syntax-directed translation

- Synthesized attributes flow from children to parent.
- Expression evaluation is a simple example.
- Bottom-up parsers handle synthesized attributes naturally.

Production with action idea:

$E \rightarrow E1 + T \quad \{ E.val = E1.val + T.val \}$

$E \rightarrow T \quad \{ E.val = T.val \}$

$T \rightarrow NUM \quad \{ T.val = NUM.value \}$

For  $3 + 4$ , reductions compute values upward.

# Creating AST nodes in actions

## PART 7 • Syntax-directed translation

- Actions can allocate nodes with operator and child pointers.
- The parser returns a pointer to the root AST node.
- Later phases walk the tree for type checking and code generation.

```
expr : expr '+' term { $$ = newBinaryNode('+', $1, $3); }  
    | term           { $$ = $1; }  
    ;  
  
factor : NUM          { $$ = newNumberNode($1); }  
      | ID            { $$ = newIdNode($1); }  
      | '(' expr ')' { $$ = $2; }  
      ;
```

# Symbol-table hooks during parsing

## PART 7 • Syntax-directed translation

- Declarations can insert names into the current scope.
- Identifier uses can create AST nodes and later resolve against symbol tables.
- Blocks can push and pop scope environments.
- Many compilers delay full checks to semantic analysis for cleaner design.

### Parser and symbol table

Use the example on the left to trace parser behavior.

Input

Stack

Action

# AST design choices

## PART 7 • Syntax-directed translation

- Keep nodes close to language concepts: IfStmt, WhileStmt, BinaryExpr, CallExpr.
- Store source locations for good diagnostics.
- Do not preserve unnecessary punctuation unless it matters for error messages or pretty printing.
- Separate parsing structure from semantic meaning where possible.

### AST design checklist

- 1 Language-level nodes
- 2 Source locations
- 3 Operator/operand fields
- 4 Scope boundaries
- 5 Clean traversal APIs

# What makes a good syntax error message?

## PART 8 • Error recovery

- Shows where the parser detected the problem.
- Mentions the unexpected token and useful expected tokens.
- Avoids blaming the wrong location when the real error is earlier.
- Continues analysis to find more independent errors.

Detection → report → recover

Detect

Report

Recover

Continue

```
stmt : error ';' { yyerrok; }  
      | ID ':=' expr ';' ;
```

Good recovery reports the real error and avoids cascaded false errors.

# Panic-mode recovery

## PART 8 • Error recovery

- Skip input tokens until a synchronizing token is found.
- Common synchronizing tokens: ;, }, ), end-of-line, or FOLLOW sets.
- Simple to implement and prevents infinite loops.
- May skip a large portion of input, so messages should be cautious.

### Panic-mode recovery

Detect

Report

Recover

Continue

```
stmt : error ';' { yyerror; }  
      | ID ':=' expr ';' ;
```

Good recovery reports the real error and avoids cascaded false errors.

# Error productions in Yacc/Bison

## PART 8 • Error recovery

- The special token error can appear in grammar rules.
- The parser discards input until the rule can be completed.
- `yyerrok` resets error status after recovery.

```
stmt : ID '=' expr ';'
      | error ';'      { yyerrok; }
      ;

block : '{' stmt_list '}'
       | '{' error '}' { yyerrok; }
       ;
```

# LL synchronization with FOLLOW

## PART 8 • Error recovery

- In LL parsers, blank table cells can call recovery routines.
- For nonterminal A, tokens in FOLLOW(A) are common synchronization points.
- If lookahead is in FOLLOW(A), pop A and continue.
- Otherwise skip tokens until a useful point is found.

### FOLLOW-based synchronization

| Symbol | FIRST             | FOLLOW       |
|--------|-------------------|--------------|
| E      | { id, ( }         | { \$, ) }    |
| E'     | { +, $\epsilon$ } | { \$, ) }    |
| T      | { id, ( }         | { +, \$, ) } |
| T'     | { *, $\epsilon$ } | { +, \$, ) } |

FIRST predicts what can begin; FOLLOW tells what may legally come after.

# Avoiding cascaded errors

## PART 8 • Error recovery

- After one error, parser state may be unreliable.
- Do not print the same error repeatedly while skipping tokens.
- Suppress messages until parser has consumed a reasonable amount or reached synchronization.
- Good recovery improves student compiler projects significantly.

### Control error cascades

Detect

Report

Recover

Continue

```
stmt : error ';' { yyerrok; }  
      | ID ':=' expr ';' ;
```

Good recovery reports the real error and avoids cascaded false errors.

# Mini case: expression grammar for LL(1)

## PART 9 • Case studies and practice

- Start with left-recursive expression grammar.
- Eliminate left recursion for top-down parsing.
- Compute FIRST/FOLLOW for transformed grammar.
- Build table and trace an input such as `id + id * id`.

### LL(1) mini project

- 1 Transform grammar
- 2 Compute sets
- 3 Build table
- 4 Trace stack
- 5 Compare AST shape

# LL(1) expression grammar after transformation

## PART 9 • Case studies and practice

- This grammar is suitable for predictive parsing.
- $E'$  and  $T'$  model repetitions of  $+$  and  $*$  groups.
- AST actions must rebuild left associativity if required.

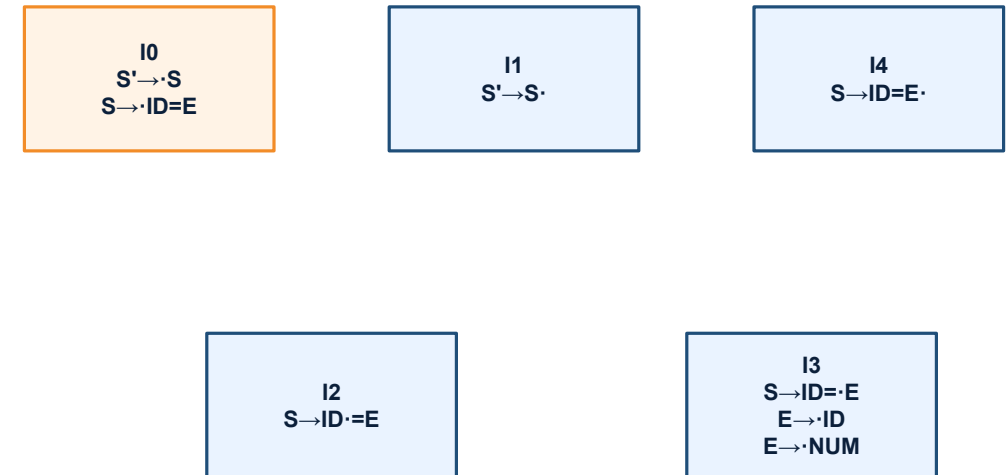
```
E  → T E'
E' → + T E' | - T E' | ε
T  → F T'
T' → * F T' | / F T' | ε
F  → ( E ) | id | num
```

# Mini case: LR automaton for assignment

## PART 9 • Case studies and practice

- Use a small grammar to practice LR item construction.
- Build closure and goto states manually.
- Observe where reductions become available.
- Then convert item sets into ACTION/GOTO entries.

### Assignment grammar DFA sketch



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# Assignment grammar for LR practice

## PART 9 • Case studies and practice

- This grammar is intentionally small for hand construction.
- Students should create item sets and table entries.
- Then trace `ID = NUM $`.

```
S' → S
S  → ID '=' E
E  → E '+' T | T
T  → ID | NUM
```

Practice input:  
`ID = NUM + ID $`

# Exam pattern: FIRST/FOLLOW

## PART 9 • Case studies and practice

- Write productions clearly and mark nullable nonterminals first.
- Compute FIRST for RHS sequences, not just individual symbols.
- Add \$ to FOLLOW(start symbol).
- Iterate until no set changes; do not stop after one pass.

### FIRST/FOLLOW exam checklist

| Symbol | FIRST             | FOLLOW       |
|--------|-------------------|--------------|
| E      | { id, ( }         | { \$, ) }    |
| E'     | { +, $\epsilon$ } | { \$, ) }    |
| T      | { id, ( }         | { +, \$, ) } |
| T'     | { *, $\epsilon$ } | { +, \$, ) } |

FIRST predicts what can begin; FOLLOW tells what may legally come after.

# Exam pattern: LL(1) table

## PART 9 • Case studies and practice

- For each production, place it under FIRST of the RHS.
- For  $\epsilon$  productions, place under FOLLOW of the left-hand nonterminal.
- Any duplicate in a cell is a proof of not LL(1).
- Blank cells are errors, not missing work.

### LL(1) table construction

| M[A,a] | id                  | +                         | *                     | \$                        |
|--------|---------------------|---------------------------|-----------------------|---------------------------|
| E      | $E \rightarrow TE'$ |                           |                       |                           |
| E'     |                     | $E' \rightarrow +TE'$     |                       | $E' \rightarrow \epsilon$ |
| T      | $T \rightarrow FT'$ |                           |                       |                           |
| T'     |                     | $T' \rightarrow \epsilon$ | $T' \rightarrow *FT'$ | $T' \rightarrow \epsilon$ |

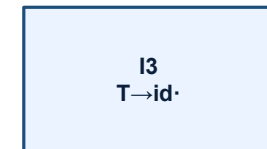
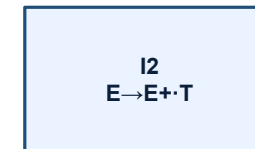
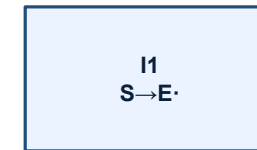
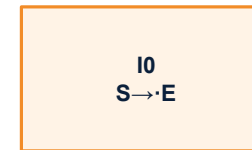
LL(1) means no table cell needs two productions.

# Exam pattern: LR item sets

## PART 9 • Case studies and practice

- Always augment the grammar first.
- Start from closure of  $S' \rightarrow \cdot S$ .
- Label transitions by the symbol after the dot.
- Do not forget closure after moving the dot.
- Complete items create reductions, not shifts.

### LR item-set checklist



LR states are sets of items. Edges are GOTO transitions over grammar symbols.

# Final summary

## PART 9 • Case studies and practice

- Parsing turns token streams into grammatical structure.
- Top-down parsing predicts; bottom-up parsing reduces.
- FIRST/FOLLOW power LL(1) and SLR ideas.
- LR item automata are the heart of shift-reduce parser generators.
- Yacc/Bison connects grammar rules with semantic actions and Lex/Flex scanners.

### Five takeaways

- 1 CFG defines syntax
- 2 LL predicts
- 3 LR reduces
- 4 Automata build tables
- 5 Actions build AST

# Suggested classroom exercises

## PART 9 • Case studies and practice

- Use these as tutorials, assignments or viva preparation.
- Require students to show traces, not just final answers.
- Ask them to identify the exact reason for conflicts.

1. Remove left recursion from an expression grammar.
2. Compute FIRST/FOLLOW for a statement grammar.
3. Build LL(1) table and trace `id + id * id`.
4. Construct LR(0) item sets for  $S \rightarrow C C$ ,  $C \rightarrow c C \mid d$ .
5. Build an SLR table and mark reductions using FOLLOW.
6. Write a Flex+Bison calculator with precedence.
7. Add error productions for missing semicolon recovery.

# Reference orientation

## PART 9 • Case studies and practice

- Core concepts follow standard compiler-design treatment used in Aho, Sethi & Ullman / Aho & Ullman texts.
- This deck paraphrases the concepts and uses original classroom examples/diagrams.
- For deeper study, read compiler chapters on context-free grammars, top-down parsing, LR parsing, syntax-directed translation and parser generators.
- Recommended lab pairing: Flex scanner deck first, then this parsing deck.

### From scanner deck to parser deck

Tokens

Parser

AST

Semantic

Parser accepts only token streams that can be generated by the programming-language grammar.

```
if (x < 10) y = y + 1;
```

```
ID ( ID < NUM ) ID = ID + NUM ;
```